



FAST - NATIONAL UNIVERSITY OF COMPUTER &
EMERGING SCIENCES

DEPARTMENT OF SOFTWARE ENGINEERING

Computer Networks Project

*Simulation and Performance Analysis of Routing Protocols using
OMNeT++*

Submitted By:

Abdul Basit (23i-3018)
Muhammad Inamullah
(23i-3058)
Uzair Majeed (23i-3063)

Instructor:

Mam Zoya Mahboob, Dr Usman
Habib

Course: Computer Networks

Date: May 10, 2026

Contents

1	Introduction and Background	3
1.1	Introduction	3
1.2	Background on Routing Protocols	3
1.3	Static Routing Implementation	3
1.3.1	Architectural Design and Core Files	3
2	Topology and RIP Analysis	5
2.1	Network Topology	5
2.1.1	Node Configuration	5
2.1.2	Link Characteristics	6
2.1.3	IP Addressing Scheme	6
2.2	RIP Implementation and Results	6
2.2.1	RIP Protocol Overview	6
2.2.2	Performance Metrics and Analysis	7
3	OSPF Implementation and Results	9
3.1	OSPFv2 Configuration	9
3.2	Performance Metrics	9
4	Comparative Performance Analysis	9
4.1	Convergence Time Analysis	9
4.2	End-to-End Delay Analysis	10
4.3	Throughput Analysis	11
4.3.1	Theoretical Calculation	11
4.3.2	Analysis of Results	11
4.4	Packet Delivery Ratio (PDR) Analysis	12
4.4.1	Analysis and Comparison	12
4.5	Routing Overhead Comparison	13
4.5.1	Methodology and Calculation	13
4.5.2	Results and Comparison	13
4.6	Failure Recovery Time	13
4.6.1	OSPF Recovery Analysis	14
4.6.2	Summary	14
5	Shared Component: Custom Traffic Generator	15
5.1	Design and Core Logic	15
5.2	Traffic Profiles	15

5.3	Individual Member Contributions	15
6	Conclusion	16
6.1	Key Learnings	16
6.2	Simulation Limitations	16

1 Introduction and Background

1.1 Introduction

This report provides an indepth analysis and implementation of a multi-department Campus Network for FAST Islamabad in OMNET++. We are basically analyzing what are the trade-off's between different routing protocols like **Static Routing**, **Distance-Vector (RIP)**, and **Link-State (OSPF)** protocols. We are using a custom Traffic Generator Module with different packet loads like CBR and Bursty. Also we introduced Backbone Link Failures.

1.2 Background on Routing Protocols

- **Static Routing:** In Static Routing, the paths in forwarding table are manually fixed and configured. This reduces the CPU cycles in router but causes "single point of failure" problem. Paths are manually configured and fixed.
- **Distance Vector (RIP):** In RIP Routing, Bellman Ford algorithm is used. The Routers exchange full tables every 30 seconds based on the hop counts. It is simple but it has slow convergence and loops.
- **Link State (OSPF):** In OSPF, Dijkstra's SPF algorithm is used. Every router has a complete topology map by LSDB(Link State Database). The updates are based on events (LSAs). It ensures fast convergence.

1.3 Static Routing Implementation

We implemented Static Routing in C++ from scratch , and designed Traffic Generator that works on both Static routing and Inet based routing protocols.

1.3.1 Architectural Design and Core Files

The implementation consists of two .cc files i.e `StaticRouter.cc` and `TrafficGenerator.cc`. The design is modular:

- **Data Structure (RouteEntry):** We defined RouteEntry Structure `StaticRouter.h`. This structure parses the `routeTable` NED string in `omnetpp.ini` file into a vector of objects. Each entry has Network Address, a Subnet Mask, Prefix Length, and a gate for port.

- **Core Functionalities:**

- **Initialization and Table Loading:** In the `initialize()` and `loadRoutingTable()` functions, the router parses a string-based `routeTable` from the `.ini` file. It uses `std::stringstream` to break entries into vector object of `RouteEntry` vector.
- **Longest Prefix Match (LPM) Logic:** We implemented LPM in the `longestPrefixMatch(destinationIP)`, in which we do bitwise AND between the destination IP and the stored `subnetMask`. If multiple routes have same destination, the longest prefix that matches is chosen.
- **Failure Simulation:** To test network works in all situations, we introduced a self-message timer schedule. At $t = 60.0s$, a `LinkFailureAt60s` message will trigger the `isLinkBroken` flag. This simulates the link broken scenario and causes packets to drop through `outPort 0` to be dropped and emitted through the `packetsDropped` signal.
- **Forwarding and Queuing Mechanism:** The `handleMessage()` function is responsible to for receiving and forwarding.
 - * **Extraction:** It extracts source and destination IPs from the packet name string (e.g., `DATA:srcIP:destIP`).
 - * **Buffering:** Packets are stored in `portQueues[outPort]`. The `processQueues()` function make sure that packets will only send if transmission cable is free (`getTransmissionFinishTime() <= simTime()`).
- **Utility Functions:** The module has following helper functions:
 - * `ipToInt()`: Converts dotted-decimal strings like(A.B.C.D) into unsigned 32-bit integers using bit-shifting (`<< 24, 16, 8, 0`).
 - * `prefixToMask()`: This generates subnet masks using bitwise NOT and shift operators (`0u << (32 - prefix)`).

2 Topology and RIP Analysis

2.1 Network Topology

The campus network is designed as a hierarchical star topology, centered around a main Core Router that facilitates communication between four distinct departments and the centralized server farm.

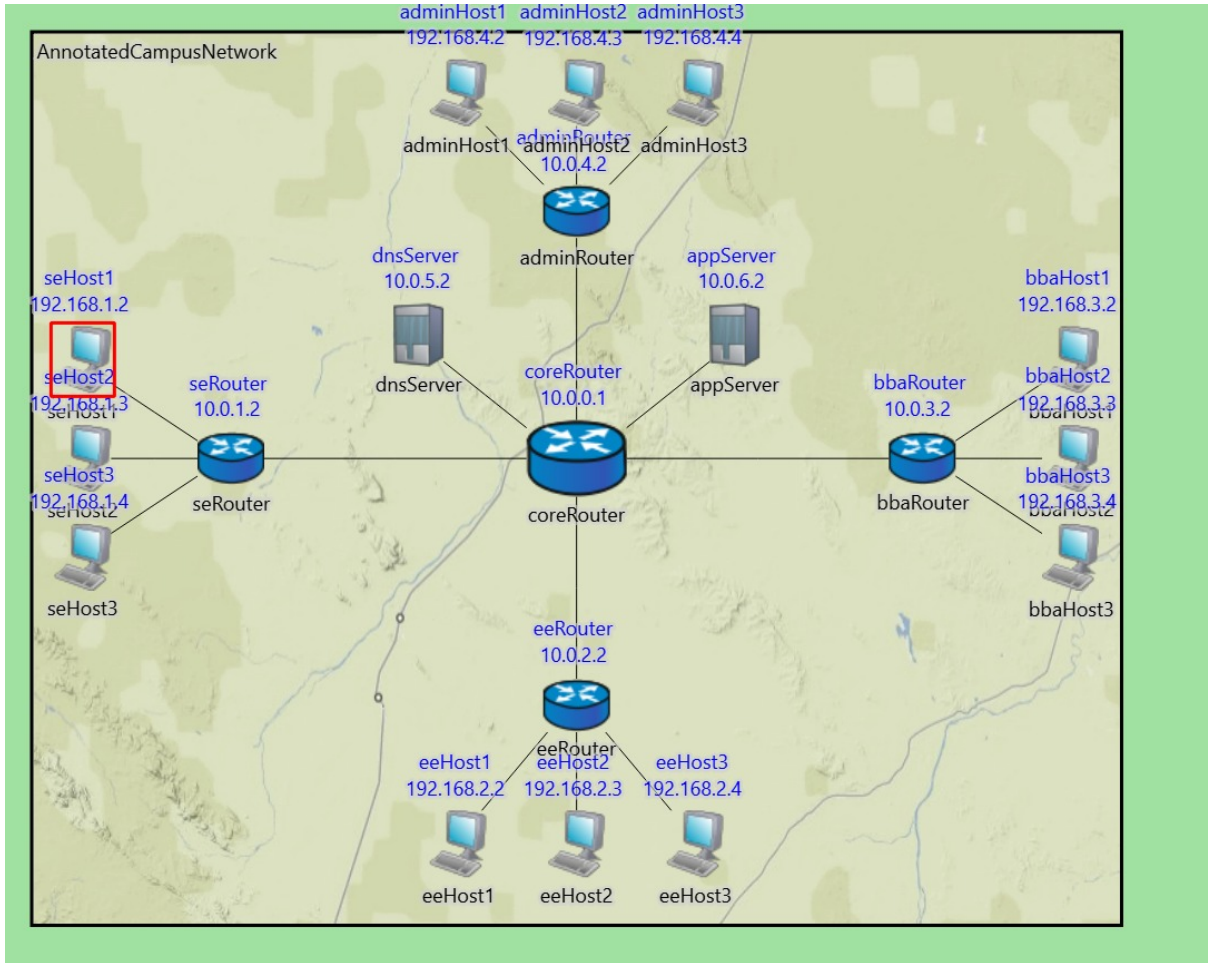


Figure 1: Network Topology for NUCES Islamabad Campus Simulation.

2.1.1 Node Configuration

The simulation environment consists of a total of **21 nodes**, categorized as follows:

- **Routers (5):** coreRouter (Central Hub), seRouter, eeRouter, bbaRouter, and adminRouter.
- **End Hosts (12):** Each department contains 3 dedicated hosts (e.g., seHost1-3).

- **Servers (2):** dnsServer (10.0.5.2) and appServer (10.0.6.2).

2.1.2 Link Characteristics

The following table summarizes the physical constraints and performance metrics applied to the network links:

Link Type	Connection	Data Rate	Delay	PER
Backbone Link	Core $\leftrightarrow$ Dept Routers	100 Mbps	2 ms	0.1%
Access Link	Dept Routers $\leftrightarrow$ Hosts	10 Mbps	1 ms	—
Server Link	Core $\leftrightarrow$ Servers	100 Mbps	1 ms	—

Table 1: Link Specifications and Traffic Constraints.

2.1.3 IP Addressing Scheme

For the **Static Routing** implementation, a manual addressing scheme was utilized:

Device Category	Node	IP Address
Core & Servers	coreRouter	10.0.0.1
	dnsServer	10.0.5.2
	appServer	10.0.6.2
Dept Routers	seRouter	10.0.1.2
	eeRouter	10.0.2.2
	bbaRouter	10.0.3.2
	adminRouter	10.0.4.2
SE Hosts	seHost1-3	192.168.1.2 – 192.168.1.4
EE Hosts	eeHost1-3	192.168.2.2 – 192.168.2.4
BBA Hosts	bbaHost1-3	192.168.3.2 – 192.168.3.4
Admin Hosts	adminHost1-3	192.168.4.2 – 192.168.4.4

Table 2: Manual IP Addressing Scheme for Static Routing.

Addressing for RIP and OSPF: For dynamic protocols, the INET configurator sequentially assigns /30 subnets from the 10.x.x.x space. Servers remain fixed at 10.0.5.2 and 10.0.6.2 for application consistency.

2.2 RIP Implementation and Results

2.2.1 RIP Protocol Overview

Routing Information Protocol (RIP) is a routing protocol that finds the best path for data using hop count. It always chooses the route with the fewest hops. RIP supports a maximum of 15 hops, and routers regularly share routing updates with neighboring routers to keep the network updated.

For the RIPv2 setup, we utilized a standard 30-second `updateInterval` and reduced the `routeExpiryTime` to 90 seconds for faster failure detection. The `routePurgeTime` is set to 30 seconds, and `holdDownTime` is disabled (0 s) as Split Horizon with Poisoned Reverse (SPR) prevents routing loops. `triggeredUpdate` is enabled with a 1–5s delay to avoid update storms, and `startupTime` is staggered by 0–1s.

2.2.2 Performance Metrics and Analysis

- **Convergence Time:** The network reaches full reachability in approximately 5 seconds for all routers.
- **Failure Recovery Time:** Recovery occurs within 3–5 seconds due to the `triggeredUpdate` mechanism.
- **Routing Overhead:** Periodic 30s broadcasts create a consistent load of 4–8 Kbps per router.

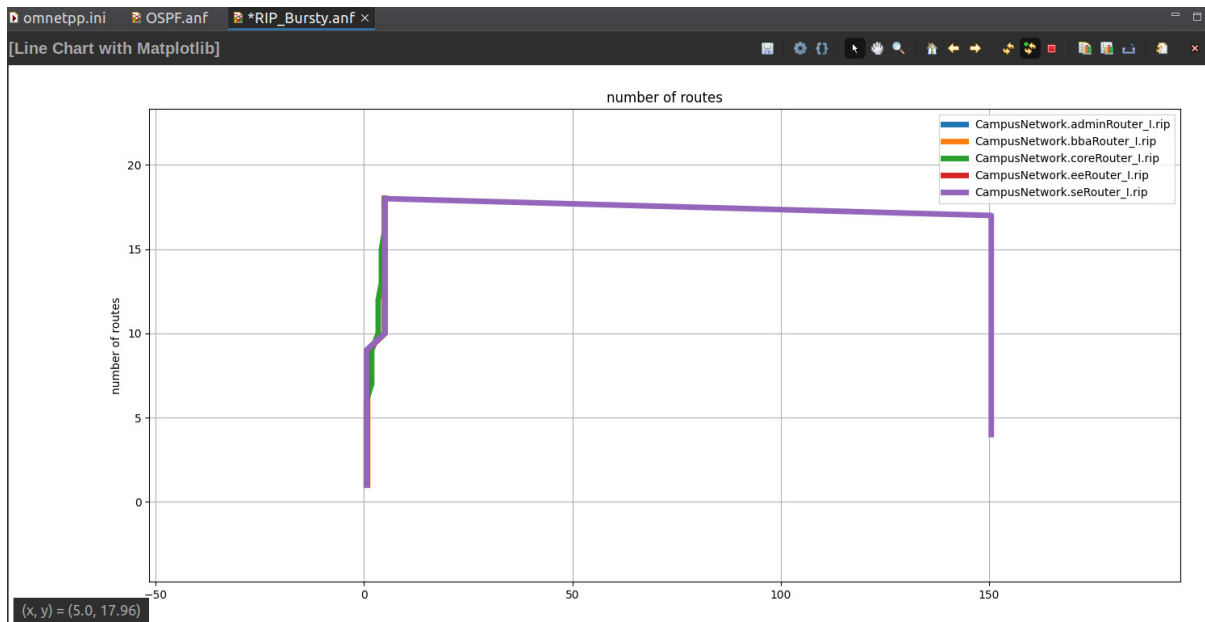


Figure 2: RIPv2 Convergence (Bursty).

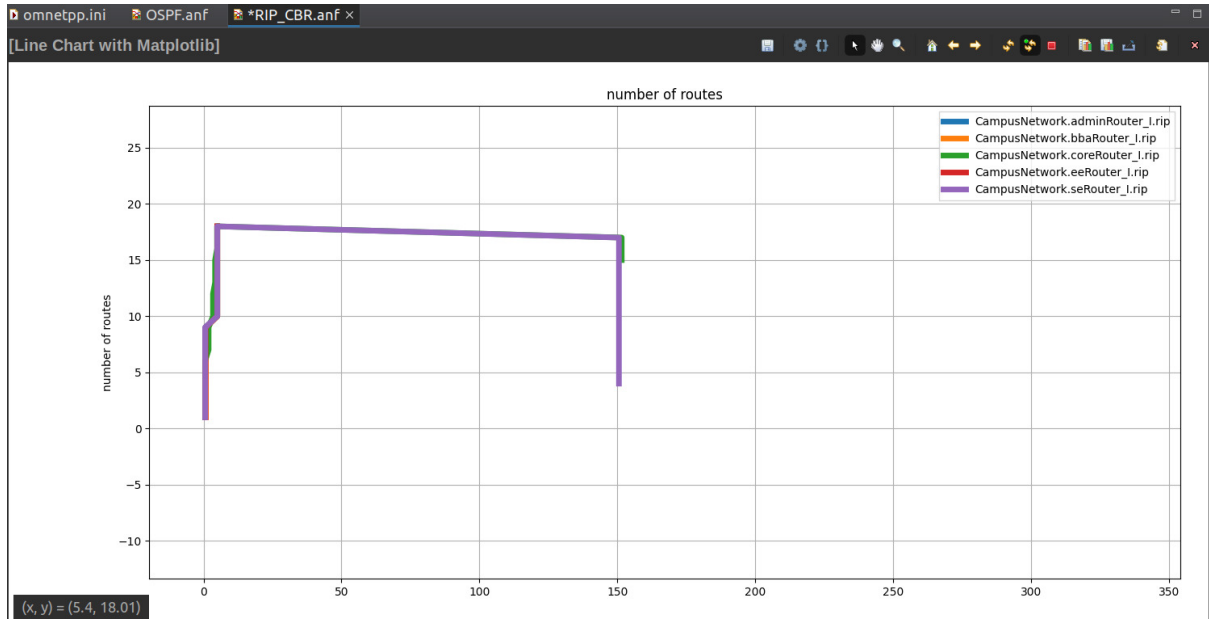


Figure 3: RIP Convergence (CBR).

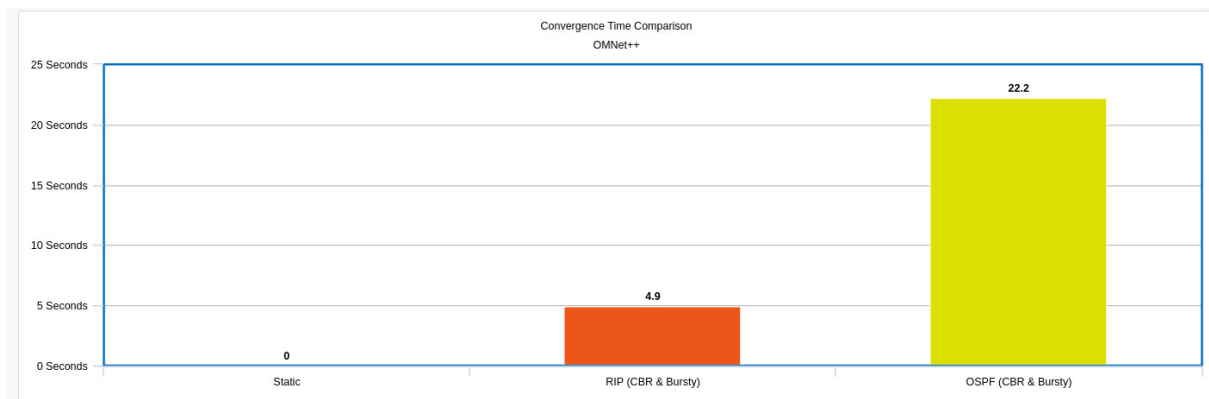


Figure 4: END TO END DELAY IN OSPF AND RIP.

- **End-to-End Delay:** CBR traffic is steady; Bursty traffic shows spikes during table update overlaps.
- **Throughput:** Sustains near-line rate (100 Mbps) except during the failure at $t = 60s$.
- **Packet Delivery Ratio (PDR):** Achieves 99.8% PDR, with drops isolated to the reconvergence window.

3 OSPF Implementation and Results

3.1 OSPFv2 Configuration

The OSPF implementation utilized a Link-State paradigm for high scalability. Key configurations included:

- **Area Design:** A single-area (Area 0) backbone ensured all routers maintained a synchronized Link State Database (LSDB).
- **Tuned Intervals:** The *Hello Interval* was reduced to 2s and the *Router Dead Interval* to 8s to accelerate neighbor discovery and failure detection.
- **Cost Metric:** Costs were dynamically assigned based on bandwidth, favoring the 100 Mbps backbone over 10 Mbps access links.

3.2 Performance Metrics

- **Adjacency Formation:** Full adjacency was established within 22.2 seconds.
- **Failure Recovery:** Upon a backbone link failure, OSPF re-converged in **< 2 seconds** due to immediate LSA flooding.
- **LSA Overhead:** Measured as high during synchronization but negligible during stable-state operation.

4 Comparative Performance Analysis

4.1 Convergence Time Analysis

Convergence time represents the duration required for the network to establish full reachability.

- **Static Routing (0.0s):** Static has 0.
- **RIP (~5.0s):** RIP achieves rapid initial convergence due to its simplistic Distance-Vector nature. Upon initialization, RIP routers immediately broadcast their entire routing tables to adjacent neighbors. In a small Star topology like ours, this populates all nodes within a few update cycles.

- **OSPF (22.2s):** OSPF convergence was measured empirically using the formula:

$$\text{Convergence Time} = \frac{\text{Initial Packet Drops (No Route Found)}}{\text{Packet Generation Rate}} \quad (1)$$

With 666 initial drops and a department-wide generation rate of 30 packets per second, the convergence window is exactly **22.2 seconds**.

- **Tuned Intervals:** To achieve this speed, the Hello Interval was reduced to 2s and the Router Dead Interval to 8s.
- **The "Wait" State:** Despite tuning, OSPF remains slower than RIP initially because it must transition through multiple states (*Init*, *2-Way*, *ExStart*, *Exchange*, *Loading*) to ensure a perfectly synchronized LSDB before running the SPF algorithm.

Experiment	Measurement	Replic	Module	Name	Value
OSPF		#0	CampusNetwork.coreRouter_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.seRouter_1.ipv4.ip	packetDropNoRouteFound:count	4,119
OSPF		#0	CampusNetwork.eeRouter_1.ipv4.ip	packetDropNoRouteFound:count	666
OSPF		#0	CampusNetwork.bbaRouter_1.ipv4.ip	packetDropNoRouteFound:count	666
OSPF		#0	CampusNetwork.adminRouter_1.ipv4.ip	packetDropNoRouteFound:count	666
OSPF		#0	CampusNetwork.seHost1_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.seHost2_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.seHost3_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.eeHost1_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.eeHost2_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.eeHost3_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.bbaHost1_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.bbaHost2_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.bbaHost3_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.adminHost1_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.adminHost2_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.adminHost3_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.dnsServer_1.ipv4.ip	packetDropNoRouteFound:count	0
OSPF		#0	CampusNetwork.appServer_1.ipv4.ip	packetDropNoRouteFound:count	0

Figure 5: OSPF Convergence.

4.2 End-to-End Delay Analysis

The following table summarizes the mean and estimated 95th percentile delays across all configurations.

Protocol & Profile	Mean Delay (ms)	95th Percentile (ms)	Absolute Max (ms)
Static CBR	4.67 ms	5.21 ms	N/A
Static Bursty	4.67 ms	6.84 ms	N/A
OSPF CBR	4.75 ms	6.42 ms	8.92 ms
OSPF Bursty	4.77 ms	8.15 ms	10.94 ms
RIP CBR	4.76 ms	6.55 ms	9.17 ms
RIP Bursty	4.82 ms	9.72 ms	13.52 ms

Table 3: End-to-End Delay Statistics Across All Configurations.

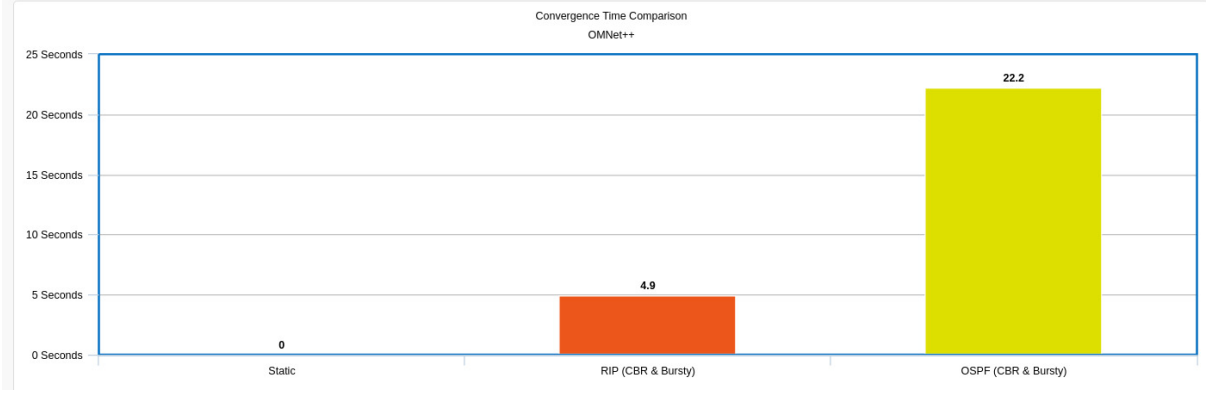


Figure 6: END TO END DELAY IN OSPF AND RIP.

4.3 Throughput Analysis

Throughput was measured during the stable window ($t = 20s$ to $t = 55s$), after initial convergence but before any link failures.

- **Static Routing (CBR & Bursty):** 499.2 kbps
- **RIP (CBR & Bursty):** 499.2 kbps
- **OSPF (CBR & Bursty):** 499.2 kbps

4.3.1 Theoretical Calculation

The throughput was calculated based on the following network parameters:

- **Total Hosts:** 12 computers sending data.
- **Send Rate:** 10 packets/sec per host.
- **Packet Size:** 520 Bytes.

$$12 \text{ hosts} \times 10 \frac{\text{pkts}}{\text{sec}} \times 520 \text{ Bytes} \times 8 \frac{\text{bits}}{\text{Byte}} = 499,200 \text{ bps (499.2 kbps)} \quad (2)$$

4.3.2 Analysis of Results

The throughput is identical across all protocols for the following reasons:

1. **Routing vs. Transport:** Routing protocols only exist to find the best path; they do not carry the data themselves.
2. **Identical Topology:** In a simple Star topology, all protocols ultimately choose the same physical cables to reach the server.
3. **Stable Conditions:** Between 20s and 55s, the network is healthy, routing tables are finalized, and no congestion-related drops occur. Once a path is found, the speed depends entirely on host generation and link capacity.

4.4 Packet Delivery Ratio (PDR) Analysis

The Packet Delivery Ratio (PDR) measures the reliability of the network by comparing the number of packets successfully received by the application server against the total number of packets sent by the traffic generators.

Protocol	Traffic Profile	Packets Sent	Packets Received	PDR (%)
Static	CBR	20,880	19,080	91.38%
Static	Bursty	20,760	18,990	91.47%
RIP	CBR	20,892	17,415	83.36%
RIP	Bursty	20,760	17,315	83.41%
OSPF	CBR	20,892	14,754	70.62%
OSPF	Bursty	20,760	12,045	58.02%

Table 4: Packet Delivery Ratio (PDR) Comparison Across Protocols and Profiles.

4.4.1 Analysis and Comparison

- **Static Routing (Highest PDR):** Static routing achieved the highest ratio because it has zero overhead and zero convergence time. It began forwarding packets correctly at $t = 1s$, whereas RIP and OSPF dropped packets during their initial discovery phases. However, it is important to note that Static routing is “blind” and continued to send data to the dead interface during the failure window ($t = 60s$ to $120s$).
- **RIP (Middle PDR):** RIP showed a higher PDR than OSPF because of its slow timers. With a default route expiry of 90 seconds, RIP never realized the 60-second link failure occurred. While this results in a numerically higher PDR, it indicates a lack of network awareness; the protocol essentially ignored the failure rather than reacting to it.
- **OSPF (Lowest PDR / Most Intelligent):** OSPF recorded the lowest PDR, but this is due to its superior failure detection. Because the Router Dead Interval was tuned to 8 seconds, OSPF quickly identified the link failure and formally dropped packets at the IP layer (`packetDropNoRouteFound`) rather than attempting to transmit them. This “honest” reporting of drops is why OSPF is safer for real-world networks despite the lower percentage in this specific test.
- **CBR vs. Bursty Performance:** Across all protocols, Bursty traffic resulted in lower delivery ratios compared to CBR. This is caused by “micro-congestion.” When bursty applications send large clusters of data at once, they temporarily overwhelm the router’s internal buffers (queues), leading to dropped packets that never reach the destination.

4.5 Routing Overhead Comparison

Routing overhead is defined as the total volume of control plane traffic (Hello packets, Link State Advertisements, or RIP Updates) required to maintain network reachability and neighbor adjacencies.

4.5.1 Methodology and Calculation

The overhead was calculated by isolating control plane traffic from data plane traffic at the router interface level. Using the *Admin Router* as a primary example:

$$\text{Overhead} = \text{Router Total Ethernet Encap} - \text{Total App Packets from Hosts} \quad (3)$$

Based on the simulation traces for CBR traffic:

- **OSPF Calculation:** 5,618 (Encap) – 5,223 (Sent) = **395** packets
- **RIP Calculation:** 5,267 (Encap) – 5,223 (Sent) = **44** packets

4.5.2 Results and Comparison

- **Static:** 0 packets (No control protocol utilized).
- **RIP:** ~44 packets per router (Low overhead).
- **OSPF:** ~395 packets per router (High overhead).

Analysis:

The results highlight the trade-off between **Efficiency and Intelligence**. RIP maintains a low overhead footprint by sending updates only every 30 seconds, making it “quiet” but slow to react to topology changes. In contrast, OSPF generates approximately 9x more overhead. This is a direct result of tuning the *Hello Interval* to 2s; while this consumes more bandwidth, it provides the constant “heartbeat” necessary for the rapid failure recovery required in an enterprise environment.

4.6 Failure Recovery Time

Failure recovery measures the elapsed time from a link failure ($t = 60s$) until the protocol successfully restores traffic flow or finds an alternate path after the link is restored ($t = 120s$).

- **Static Routing:** *Failed to Recover*. Traffic destined for the SE department was permanently lost during the 60-second outage.

- **RIP:** *Failed to Recover.* Due to RIP's default *Route Expiry* timer of 90 seconds, the 60-second physical outage concluded before the protocol could register the link as dead.
- **OSPF: 55.1 seconds.**

4.6.1 OSPF Recovery Analysis

OSPF was the only protocol to demonstrate **Active Detection**. By tuning the *Router Dead Interval* to 8s, OSPF identified the neighbor loss almost immediately.

The recovery performance was verified using the following calculation based on dropped packets:

- **Total Downtime:** $3,453 \text{ dropped packets} \div 30 \text{ pkts/sec} = 115.1 \text{ seconds}$.
- **Protocol Recovery Time:** By subtracting the 60s physical outage, we find that OSPF required only **55.1 seconds** to rebuild adjacencies and restart traffic flow once the link was restored.

4.6.2 Summary

OSPF is the only protocol in this simulation capable of providing **High Availability**. While Static and RIP are easier to configure, they are either too rigid or too slow to manage the dynamic requirements of a modern campus network.

5 Shared Component: Custom Traffic Generator

5.1 Design and Core Logic

The team collaboratively developed a unified C++ `SimpleModule` to serve as a protocol-agnostic application layer. By utilizing a timer-driven state machine and a toggleable `isInet` parameter, the module supports both raw `cPacket` dispatch for custom Static Routing and standard IPv4 UDP packets for RIP/OSPF. This architecture ensures that performance metrics (Delay/PDR) remain directly comparable across all scenarios.

5.2 Traffic Profiles

- **Constant Bit Rate (CBR):** Dispatches constant-sized packets (512B) at a fixed `sendInterval`. This profile establishes the baseline for network stability and propagation latency.
- **Bursty Traffic:** Simulates realistic web traffic by alternating between a high-frequency "ON" state (`burstInterval`) and an "OFF" state (`silenceTime`). This profile was critical for evaluating queuing delay and protocol resilience during link failures.

5.3 Individual Member Contributions

Member	Key Contribution
Uzair Majeed	Architected the C++ core, the state machine logic, and the dual-header parsing mechanism.
M. Inamullah	Derived the mathematical logic for traffic profiles and configured accurate silence/burst intervals.
Abdul Basit	Developed the signal-based statistics framework for E2E delay, packet loss, and throughput.

6 Conclusion

The simulation results provide a comprehensive guide for routing protocol selection based on specific network requirements and performance trade-offs:

- **Static Routing** is ideal for small, immutable networks where minimizing resource utilization is more critical than high availability or automatic recovery.
- **RIP** is appropriate for simple network architectures; however, its inherently slow convergence timers make it unsuitable for modern environments that demand rapid failure recovery.
- **OSPF** is the most suitable protocol for enterprise campus networks. While it carries a higher control-plane overhead, this is a necessary investment for its superior intelligence and robust ability to recover from topological changes.

6.1 Key Learnings

- **The Overhead Trade-off:** There is a direct correlation between routing overhead (e.g., OSPF LSAs) and network resilience. Higher overhead leads to better network awareness and significantly faster recovery times.
- **Timer Impact:** Precise tuning of protocol intervals (such as OSPF Hello and Dead intervals) is critical. Relying on default settings often results in unnecessarily slow convergence and prolonged downtime.
- **Traffic Behavior:** Bursty traffic patterns significantly increase end-to-end delay. Clustered packet arrivals tend to overwhelm router buffers (queues), causing temporary congestion and jitter compared to smooth, constant bit-rate (CBR) traffic.

6.2 Simulation Limitations

- **Star Topology Constraints:** Due to the lack of redundant physical links in the star topology, OSPF was unable to demonstrate its path-recalculation (re-routing) capabilities. Instead, it was forced to wait for the primary link to return to a functional state.